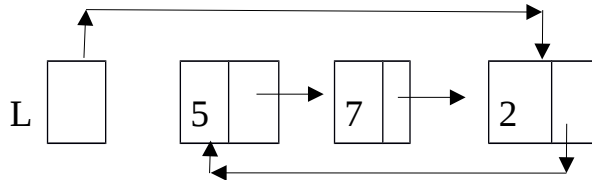


ESTRUCTURAS DE DATOS Y ALGORITMOS

JUNIO 2016

1. Multiplicar lista (2,5 puntos)



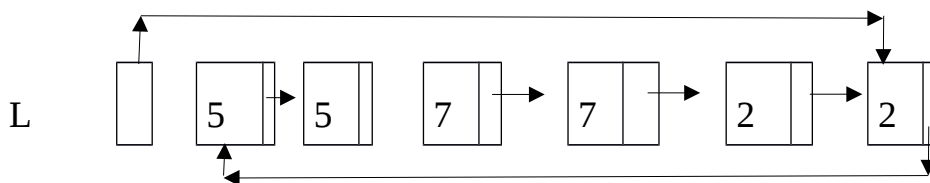
Dada una lista circular donde se accede al último elemento de la lista, se pide implementar el siguiente método:

```
public Node<T> {
    String data;
    Node<T> next;
}

public class Lista<T> {
    Node<T> last;

    public void multiplicar(Integer n) {
        // pre:  n >= 1
        // post: la lista ha sido modificada y contiene cada elemento de la
        //       lista inicial repetido "n" veces
    }
}
```

Dada la lista del ejemplo anterior, la llamada a *multiplicar(2)* obtendría como resultado la siguiente lista:



- Implementar el método multiplicar
- Calcular el coste del algoritmo, **de manera razonada**

2. Juego de bloques (2,5 puntos)

```
public class Bloque {
    int puntos;
    int salto; // valor entre -3 y +3
}

public class Juego {
    Stack<Bloque>[] tablero; // array de pilas
    public static int NUMCOLUMNAS = 7;

    public Juego() { // constructora
        tablero = (Stack<Bloque>[]) new Stack[NUMCOLUMNAS];
        for (int i = 0; i <= NUMCOLUMNAS - 1; i++) {
            tablero[i] = new Stack<Bloque>();
        }
        // código para "llenar" aleatoriamente las pilas de "bloques"
    }

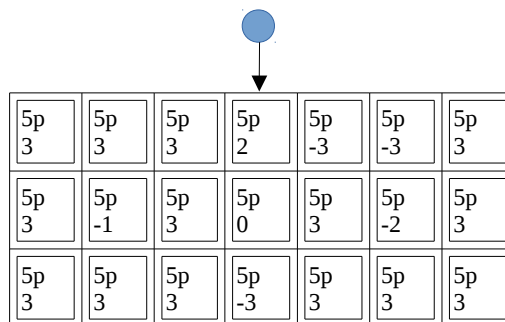
    public int jugar() {
        // Pre: el juego ha sido inicializado (se han generado los
        // bloques de inicio)
        // Post: se ha jugado una partida, en la que la bola empieza
        // cayendo encima de la columna de en medio.
        // El resultado será el número de puntos obtenido al jugar con esa bola,
        // en caso de que el juego haya sido superado, y -1 en caso contrario
    }
}
```

Esas definiciones de tipos de datos sirven para representar un juego en el que una bola va “rompiendo” bloques. Cuando la bola cae sobre una columna, entonces “rompe” el bloque que se encuentre encima de esa columna. La bola se dirigirá hacia la siguiente columna indicada por el valor del bloque que ha sido eliminado, donde la dirección puede ser derecha o izquierda, y el desplazamiento es un valor entre -3 y 3, que indica el número de columnas que se saltan en la dirección indicada. El juego acabará por dos motivos:

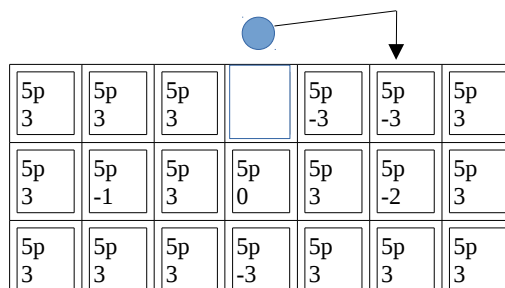
- La bola cae sobre una columna que no tiene bloques. En este caso el juego ha sido superado.
- La bola cae fuera de las columnas, indicando que se ha perdido el juego.

El siguiente gráfico muestra un ejemplo del juego:

Situación inicial (inicialmente la bola cae sobre la columna de la mitad):



Después de que la bola cae sobre el primer bloque salta 2 posiciones a la derecha:



Ahora saltará tres posiciones a la izquierda.



5p 3	5p 3	5p 3		5p -3		5p 3
5p 3	5p -1	5p 3	5p 0	5p 3	5p -2	5p 3
5p 3	5p 3	5p 3	5p -3	5p 3	5p 3	5p 3

Ahora saltará tres posiciones a la derecha.



5p 3	5p 3			5p -3		5p 3
5p 3	5p -1	5p 3	5p 0	5p 3	5p -2	5p 3
5p 3	5p 3	5p 3	5p -3	5p 3	5p 3	5p 3

Ahora saltará dos posiciones a la izquierda.



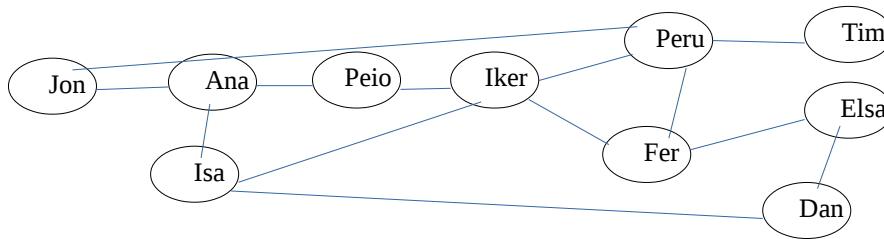
5p 3	5p 3			5p -3		5p 3
5p 3	5p -1	5p 3	5p 0	5p 3		5p 3
5p 3	5p 3	5p 3	5p -3	5p 3	5p 3	5p 3

Y así hasta que el juego termina.

Se pide implementar el procedimiento “jugar”, que jugará una partida y, al acabar el juego, devolverá el número de puntos conseguido.

3. Epidemia (2,5 puntos)

Tenemos el siguiente grafo, que representa relaciones de amistad entre personas:



El método *simularEpidemia()* servirá para medir cómo se propagará una enfermedad contagiosa cuando una persona está infectada.

Si una persona está infectada, puede contagiar esa enfermedad a sus conocidos. El método *contagio()*, perteneciente a la clase *Persona*, devolverá *true* si una persona que está en contacto con una persona infectada contraerá la enfermedad y *false* si no.

Una persona que ya ha estado en contacto con la enfermedad (puede haber contraído la enfermedad o no) no podrá ser afectada por esa enfermedad en caso de entrar en contacto con otras personas afectadas.

Se pide implementar el método *simularEpidemia()*:

```
public class Persona {
    String nombre;
    boolean infectada; // al comienzo del proceso false para todas las personas
    ArrayList<Persona> contactos;

    public void simularEpidemia()
    // pre: esta persona tiene la enfermedad
    // post: la enfermedad puede ser transmitida a los contactos de esta persona,
    //       y a sus contactos, ...

    public boolean contagio()
    // post: devuelve true si una persona en contacto con el virus se contagia
    //       y false si no
}
```

4. Obtener los N mayores (2,5 puntos)

Se tiene una lista desordenada de M valores positivos ($M = 1.000.000.000$) y se quieren obtener los N valores más grandes de la lista ($N \ll M$, por ejemplo, $N = 1.000$).

```
public ArrayList<Integer> obtenerNMejores(ArrayList<Integer> lista)
```

Se ha preguntado a 4 alumnos que nos han planteado las siguientes soluciones:

Solución 1)

```
resultado = lista vacía
repetir N veces
    buscar el mayor valor de "lista"
    meterlo en resultado
    cambiar el valor de "lista" por -1 (así no será el máximo la siguiente vez)
```

Solución 2)

```
ordenar la lista original en orden descendente
coger los N primeros y devolverlos
```

Solución 3)

```
arbolBinarioBusqueda = meter los M valores de la lista original uno a uno
repetir N veces
    buscar el mayor valor del árbol y borrarlo
    meterlo en la lista resultado
```

Solución 4)

```
arbolBinarioBusqueda = árbol vacío
repetir por cada elemento de la lista original (M veces)
    si el tamaño de arbolBinarioBusqueda es < N
        entonces añadir el elemento actual al árbol
    si no si el tamaño del árbol es = N y
        el menor valor del árbol es menor que el elemento actual
        entonces borrar menor elemento del árbol
        añadir elemento actual al árbol
devolver los elementos de arbolBinarioBusqueda
```

Se pide:

1. Ordenar los 4 algoritmos respecto a su orden de complejidad **indicando por cada uno su coste, de manera razonada.**

2. **Implementar** en Java el método que consideres más eficiente.